

# Olist E-Commerce Analytics Project

End-to-End Data Analysis: Python (Data Cleaning) • MySQL (Business Analysis) • Power BI (Dashboard)

|               |                                                                             |
|---------------|-----------------------------------------------------------------------------|
| Dataset       | Olist Brazilian E-Commerce Public Dataset                                   |
| Tools Used    | Python (pandas, matplotlib, seaborn), MySQL, Power BI                       |
| AI Assistance | ChatGPT / Claude / Gemini (used for guidance, debugging, and documentation) |
| Project Scope | Data cleaning & EDA, SQL business analysis, interactive BI dashboard        |

This report documents a self-directed data analytics project built on the public Olist Brazilian e-commerce dataset. It covers data cleaning in Python, business analysis in SQL, and dashboard visualization in Power BI. AI tools were used throughout as a coding and documentation assistant, with all logic reviewed and validated by the author.

# Contents

---

1. Project Overview
2. Tools & Tech Stack
3. Data Cleaning & Preparation (Python)
4. Business Analysis (MySQL)
5. Dashboard (Power BI)
6. Key Insights
7. Repository Structure & How to Run
8. Use of AI Tools

# 1. Project Overview

The Olist dataset contains information on ~100,000 orders placed on the Olist marketplace in Brazil between 2016 and 2018, spanning customers, orders, order items, payments, reviews, products, sellers, and geolocation. The goal of this project was to take the raw, multi-table dataset through a full analytics pipeline: clean and join the data, answer core business questions with SQL, and present the findings in an interactive Power BI dashboard.

The analysis is organized around four business questions:

- **Revenue:** How much are we making, and where does it come from?
- **Customer Experience:** How satisfied are customers, and what drives satisfaction?
- **Delivery:** Are orders arriving on time, and where are the delays?
- **Business Performance:** Which sellers, categories, or regions need attention?

# 2. Tools & Tech Stack

| Stage               | Tool                                        | Purpose                                                                             |
|---------------------|---------------------------------------------|-------------------------------------------------------------------------------------|
| Data Cleaning & EDA | Python (pandas, numpy, matplotlib, seaborn) | data cleaning, merge, and explore the raw CSV files; produce charts                 |
| Business Analysis   | MySQL                                       | Join cleaned tables and answer revenue, review, delivery, and performance questions |
| Visualization       | Power BI                                    | Build an interactive dashboard (olist.pbix) for stakeholders                        |
| Assistance          | ChatGPT / Claude / Gemini                   | Used for code review, debugging, query structuring, and documentation               |

# 3. Data Cleaning & Preparation (Python)

Cleaning and preparation was done in **Olist.ipynb** using pandas. The workflow followed these steps:

- Loaded all 8 raw CSV files (customers, geolocation, order items, payments, reviews, orders, products, sellers) plus the category translation file.
- Inspected data types and missing values table by table (orders, reviews, products) using .info() and .isnull().sum().
- Converted date columns from text/object to proper datetime types (order timestamps, review dates).
- Checked for duplicate rows and duplicate order\_id values to confirm order\_id is a reliable unique key.
- Filled missing product categories with 'unknown' rather than dropping rows, then translated category names from Portuguese to English via a left join with the translation table.
- Checked order\_items price for outliers (zero/negative prices, extreme high values) as a sanity check.
- Merged all tables step by step (order\_items → orders → customers → products → sellers → reviews) into a single wide table, deduplicating reviews to one row per order first.
- Engineered three analysis columns: is\_late\_delivery (actual vs. estimated delivery date), delivery\_delay\_days (days late/early), and freight\_ratio (shipping cost as a share of item price).

- Exported the merged, cleaned table as `olist_clean_dataset.csv` for downstream SQL and BI work.

Exploratory charts (built with matplotlib/seaborn) examined: average review score for on-time vs. late deliveries; review score against days late; review score by order-value bucket and by freight-ratio bucket; the 10 best- and worst-rated product categories (minimum 30 orders); and seller-level late-delivery rate vs. average review score, with a 'priority problem' list of sellers who combine above-median revenue with an average rating below 3.5.

## 4. Business Analysis (MySQL)

---

After cleaning, the raw tables were loaded into a MySQL database (`olist_project`), renamed to shorter names (`customers`, `geolocation`, `order_items`, `payments`, `reviews`, `orders`, `products`, `sellers`, `category_translation`), and validated with row-count and null checks before analysis. Queries were organized into four themed sections, shown below with representative examples.

### 4.1 Revenue

- Total revenue across all order items
- Monthly revenue trend
- Average order value
- Revenue by product category (top 10)
- Revenue by seller

Example — monthly revenue:

```
SELECT DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m') AS month,
ROUND(SUM(oi.price), 2) AS monthly_revenue
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY month ORDER BY month;
```

### 4.2 Customer Experience

- Average review score overall
- Review score by product category
- Review score by seller
- Review score by customer state
- Review score by order-value bucket (Low / Medium / High)

Example — review score by order-value bucket:

```
SELECT CASE WHEN order_value < 100 THEN 'Low'
WHEN order_value < 500 THEN 'Medium' ELSE 'High' END AS order_value_group,
ROUND(AVG(review_score), 2) AS average_review_score, COUNT(*) AS total_reviews
FROM (SELECT o.order_id, SUM(oi.price) AS order_value, r.review_score
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
JOIN reviews r ON o.order_id = r.order_id
GROUP BY o.order_id, r.review_score) AS order_data
GROUP BY order_value_group ORDER BY average_review_score DESC;
```

### 4.3 Delivery

- Estimated vs. actual delivery time per order
- Average delivery delay (days)

- Late-delivery percentage overall
- Delivery performance by seller
- Delivery performance by customer state

Example — late-delivery percentage:

```
SELECT ROUND(100.0 * SUM(CASE WHEN order_delivered_customer_date >
order_estimated_delivery_date THEN 1 ELSE 0 END) / COUNT(*), 2)
AS late_delivery_percentage
FROM orders
WHERE order_delivered_customer_date IS NOT NULL
AND order_estimated_delivery_date IS NOT NULL;
```

#### 4.4 Business Performance (Diagnostic Queries)

- High-revenue sellers with a low average rating (revenue > 10,000 and rating < 3)
- High-revenue product categories with a low average rating
- High-volume states with poor delivery performance (>1,000 orders and elevated delay/late rate)
- Sellers with consistently poor performance (≥20 orders, rating < 3, late rate > 20%)

Example — sellers with consistently poor performance:

```
SELECT s.seller_id, COUNT(DISTINCT o.order_id) AS total_orders,
ROUND(AVG(r.review_score), 2) AS average_rating,
ROUND(100.0 * SUM(CASE WHEN o.order_delivered_customer_date >
o.order_estimated_delivery_date THEN 1 ELSE 0 END)
/ COUNT(DISTINCT o.order_id), 2) AS late_delivery_percentage
FROM sellers s JOIN order_items oi ON s.seller_id = oi.seller_id
JOIN orders o ON oi.order_id = o.order_id
LEFT JOIN reviews r ON o.order_id = r.order_id
GROUP BY s.seller_id
HAVING COUNT(DISTINCT o.order_id) >= 20 AND AVG(r.review_score) < 3
AND (SUM(CASE WHEN o.order_delivered_customer_date >
o.order_estimated_delivery_date THEN 1 ELSE 0 END)
/ COUNT(DISTINCT o.order_id)) > 0.20;
```

## 5. Dashboard (Power BI)

---

The SQL outputs and cleaned dataset feed an interactive Power BI report (**olist.pbix**) that brings the four analysis themes together in one place for non-technical stakeholders:

- **Revenue view:** total revenue, monthly trend, top categories and sellers by revenue.
- **Customer experience view:** average review score sliced by category, seller, state, and order value.
- **Delivery view:** on-time vs. late delivery rates, average delay, and geographic breakdown.
- **Business performance view:** flags for high-revenue/low-rating sellers and categories, and high-volume/poor-delivery regions, to prioritize action.

Note: the .pbix file requires Power BI Desktop (Windows) to open and refresh; a set of static screenshots or a PDF export of the report pages can be added to the repository for viewers without Power BI installed.

## 6. Key Insights

---

- Delivery experience is one of the strongest drivers of review score — late orders score noticeably lower on average than on-time orders, and the gap widens the more days late an order is.
- A small set of sellers combine high revenue with a low average rating and/or a high late-delivery rate — these are flagged as priority problems since they carry disproportionate reputational risk.
- Late-delivery rates and average delay vary meaningfully by state, pointing to logistics/geography rather than sellers alone as a contributing factor in some regions.
- Certain product categories consistently under- or over-perform on review score even after controlling for a minimum order-volume threshold, suggesting category-specific quality or expectation issues.
- Order value and freight ratio both relate to satisfaction, which is useful for pricing and shipping-cost strategy discussions.

*Note: exact figures depend on the query outputs and are best viewed by running the notebook/SQL scripts or opening the dashboard, since this document summarizes methodology rather than reproducing every numeric result.*

## 7. Repository Structure & How to Run

---

Suggested file layout for GitHub:

```
olist-analytics-project/  
  ■■■ Olist.ipynb      # Python data cleaning & EDA  
  ■■■ Olist_Project.sql  # MySQL business analysis queries  
  ■■■ olist.pbix       # Power BI dashboard  
  ■■■ olist_clean_dataset.csv  # cleaned/merged dataset (output of notebook)  
  ■■■ Olist_Project_Report.pdf  # this report  
  ■■■ README.md
```

To reproduce the project:

- 1 Download the Olist Brazilian E-Commerce dataset (Kaggle) and place the raw CSVs locally.
- 2 Run Olist.ipynb top to bottom to clean the data and produce olist\_clean\_dataset.csv and the EDA charts.

- 3 Load the raw CSVs into a MySQL instance and run `Olist_Project.sql` to create the `olist_project` database and execute the business-analysis queries.
- 4 Open `olist.pbix` in Power BI Desktop, point it at the cleaned dataset/SQL tables, and refresh to explore the dashboard.

## 8. Use of AI Tools

---

AI assistants (ChatGPT, Claude, and Gemini) were used as coding and documentation aids throughout this project — for example, structuring pandas cleaning steps, reviewing SQL query logic, and helping draft this report. All analysis logic, query results, and conclusions were reviewed and validated against the underlying data by the author before being included in the project.